Journal of Parallel and Distributed Computing 199 (2025) 105053

Contents lists available at ScienceDirect

Journal of Parallel and Distributed Computing

journal homepage: www.elsevier.com/locate/jpdc

GPU memory usage optimization for backward propagation in deep network training

Ding-Yong Hong [a,*], Tzu-Hsien Tsai [b], Ning Wang [b], Pangfeng Liu [b], Jan-Jan Wu [a]

[a] Institute of Information Science, Academia Sinica, Taipei, Taiwan
[b] Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan

ARTICLE INFO

Keywords:
Deep learning
Dynamic programming
Memory usage optimization
Memory pressure
Checkpointing

ABSTRACT

In modern Deep Learning, it has been a trend to design larger Deep Neural Networks (DNNs) for the execution of more complex tasks and better accuracy. On the other hand, Convolutional Neural Networks (CNNs) have become the standard method for most of computer vision tasks. However, the memory allocation for the intermediate data in convolution layers can cause severe memory pressure during model training. Many solutions have been proposed to resolve the problem. Besides hardware-dependent solutions, a general methodology *rematerialization* can reduce GPU memory usage by trading computation for memory efficiently. The idea is to select a set of intermediate results during the forward phase as *checkpoints*, and only save them in memory to reduce memory usage. The backward phase recomputes the intermediate data from the closest checkpoints in memory as needed. This recomputation increases execution time but saves memory by not storing all intermediate results in memory during the forward phase. In this paper, we will focus on efficiently finding the optimal checkpoint subset to achieve the least peak memory usage during the model training. We first describe the theoretical background of the training of a neural network using mathematical equations. We use these equations to identify all essential data required during both forward and backward phases to compute the gradient of weights of the model. We first identify the *checkpoint selection* problem and propose a dynamic programming algorithm with time complexity $O(n^3)$ to solve the problem of finding the optimal checkpoint subset. With extensive experiments, we formulate a more accurate description of the problem using our theoretical analysis and revise the objective function based on the tracing, and propose an $O(n)$-time algorithm for finding the optimal checkpoint subset.

1. Introduction

1.1. The memory pressure problem in deep learning

In Deep Learning, it has been a trend to design larger Deep Neural Networks (DNNs) for the execution of more complex tasks and better accuracy in the past few years [10,24,26,29,31,22]. For example, the idea of the residual block behind ResNets [10] makes it possible to train networks with over one thousand layers. Among all kinds of models, Convolutional Neural Networks (CNN) have become the standard method for object detection and most other computer vision applications. However, the memory allocated for the intermediate data of these convolution layers and the growth of the number of model parameters has become a problem [27,30,28,25,12]. On the other hand, some research shows that a sufficiently large batch size is required for good and fast convergence [19]. What makes it worse, there is a piece of evidence that the number of parameters in the state-of-the-art neural network has doubled around every 2.4 years [5].

Many solutions have been proposed to solve the memory pressure problem [13,20,17,8,9,7,14]. Offloading moves the activations during the forward phase from the memory of GPUs to the CPU memory, and then prefetches the data back at the appropriate stage during the backward phase [20]. While manufacturers have designed high-end hardware, e.g. NVLink [17], to support efficient communication between GPU and CPU, this approach may be too expensive for some researchers since NVLink is expensive.

Besides hardware-dependent solutions, there are approaches to reducing GPU memory usage by pruning unimportant connections [8,9]. We can also reduce the size of data by reducing the precision of floating point numbers. Some research shows that we can train DNN models using 16-bit wide fixed-point number with little or no degradation in

* Corresponding author.
E-mail address: dyhong@iis.sinica.edu.tw (D.-Y. Hong).

https://doi.org/10.1016/j.jpdc.2025.105053
Received 28 December 2023; Received in revised form 26 November 2024; Accepted 5 February 2025

Available online 11 February 2025

classification accuracy [7]. More importantly, it enables larger models to fit within the given memory budget [14].

In addition to hardware-dependent solutions, we have to use some techniques to keep up with the pace of the growing memory requirement. The general methodology *trading computation for memory* [2] or *rematerialization* [6,15,11] aims to save GPU memory by not storing the activations of certain layers during the forward phase and recomputing them in batch during the backward phase. The method eliminates the memory and hardware requirement for training large-scale DNN models without loss of accuracy at the cost of a moderate increase in time on recomputing activation data. Since the ideas of rematerialization and offloading are independent, another approach is to combine the two to reduce memory usage [1]. On the other hand, for image recognition tasks, an improvement of the ResNet [10] model has been proposed to reduce memory consumption by making the activations of most layers reconstructible from the activation of the next layer. This eliminates the need to store intermediate data during the forward phase [4].

Our work will focus on efficiently finding the optimal checkpoint subset to achieve the least peak memory usage during the model training. To the best of our knowledge, most existing works are built upon the *sublinear memory cost* method proposed by Chen et al. [2] since it is effective and easy to implement. But with our extensive experiments, there is a huge difference in the peak memory usage with the checkpoint subset found by their method and by our method. On the other hand, the *arbitrary computation graph* solver proposed by Feng et al. [3] is designed to solve for the optimal checkpoint subset using the same objective function as the sublinear memory cost method. We will show that the objective function is not precise in describing the behavior of the state-of-the-art deep learning platform PyTorch and thus the algorithm does not give the optimal checkpoint subset. We will demonstrate the difference in the experiment section.

In this paper, we first describe the theoretical background of training a neural network using mathematical equations. We use these equations to identify all essential data required during both the forward and backward phases to compute the gradient of weights of the model. We summarize our analysis with a piece of pseudocode to represent the implementation of existing deep learning platforms for ease of identifying the memory pressure problem and for ease of comparison with the algorithm we will propose in the later sections. Instead of the greedy approach proposed by Chen et al., which uses some heuristic to find good results in seconds, we formulate our first optimization problem as *checkpoint selection problem* based on Algorithm 2 and propose an $O(n^3)$ algorithm for finding the optimal checkpoint subset using dynamic programming. Even more, by tracing the memory usage reported from PyTorch and comparing it with our simulated results, we revise Algorithm 2 into Algorithm 4 using the theoretical analysis of model training we have mentioned at the beginning and denote the new problem as *dynamic checkpoint selection*. We formulate an accurate objective function based on the tracing and propose an $O(n)$-time algorithm for finding the optimal checkpoint subset using dynamic programming. Through extensive experiments, we conclude that our algorithm can precisely describe the behavior of the state-of-the-art deep learning platform PyTorch.

1.2. Analysis of backward propagation

We describe the computation of a neural network model abstractly. The model has n weights $w_1, \ldots, w_n$, an input x, and the ground truth G. The training for input x consists of a *forward phase* and a *backward phase*, which will update the weights. During an epoch, the training uses a set of different inputs to train the weights. The training repeats the epoch until the weights converge.

The forward phase computes data d_i with data d_{i-1} and weight w_i with a function f_i, for i from 1 to n. Note that we define d_0 to be the input x to ease the notation in Equation (1).

$$d_i = f_i(d_{i-1}, w_i) \tag{1}$$

Then we start the *backward phase* that updates weights according to a loss function. We compare the final output from Equation (1) (d_n) with the ground truth G corresponding to input x, and compute a loss function $L(d_n, G)$. Now we compute the gradient g_i of w_i so that we can update it to minimize the loss function. Since the loss is a function of the ground truth G and d_n, and d_n is a function of d_{n-1} and w_n, and so on, we can compute the gradient g_i of w_i as a product in Equation (2) by the chain rule.

$$g_i = \frac{\partial d_i}{\partial w_i}\left(\prod_{j=i+1,n} \frac{\partial d_j}{\partial d_{j-1}}\right)\frac{\partial L}{\partial d_n} \tag{2}$$

Note that in practice we do not need to compute g_i as the product in Equation (2). Instead, let $r(i)$ be a part of the product in Equation (2) then we have the following recursion.

$$r_n = \frac{\partial d_n}{\partial d_{n-1}}\frac{\partial L}{\partial d_n} \tag{3}$$

$$r_i = \frac{\partial d_i}{\partial d_{i-1}} r_{i+1}, 1 \le i \le n-1 \tag{4}$$

Now we can rewrite Equation (2) as Equation (5).

$$g_i = \frac{\partial d_i}{\partial w_i} r_{i+1} \tag{5}$$

We now compute the gradients and update the weights *backward*. We first compute g_n from Equation (3) and (5), and update weight w_n. We then compute the rest of the gradient g_i ($1 \le i \le n-1$) from Equation (4) and (5), then update weight w_i with gradient g_i. This is called *backward propagation* in the literature. Algorithm 1 gives the pseudocode for the training.

Algorithm 1 Neural Network Training.

Require: Weights $w_1, \ldots, w_n$, input x (d_0), and the ground truth G.
Ensure: Updated $w_1, \ldots, w_n$.

```
Allocate memory for d_i, 1 ≤ i ≤ n
for i ← 1 to n do                                   ▷ Forward Phase
    Compute d_i with d_{i-1} and w_i (Equation (1))
end for

for i ← n to 1 do                                   ▷ Backward Phase
    Compute r_i (Equation (3) or (4))
    Compute g_i (Equation (5))
    Update w_i with g_i                             ▷ Update the weight
end for
```

Most of the current deep-learning platforms allocate memory for all data before the computation starts to ensure memory is available during the entire training. For example, Algorithm 1 will allocate memory for all d_i, $1 \le i \le n$, so that the backward phase can proceed without allocating any more memory. However, this allocation causes severe and unnecessary memory pressure, since the backward phase proceeds in layers and does not need all d_i simultaneously.

Chen et al. [2] proposed a *checkpoint* technique that reduces memory pressure. The checkpoint technique computes all the data d_i during the forward phase just as in Algorithm 1, but only keeps a subset (called checkpoints) in memory. During the backward phase, we can *recompute* the data d_i between two checkpoints so that the backward phase can proceed. For ease of explanation, we will denote these non-checkpoints between two checkpoints as a *segment*. It is easy to see that once we complete the backward phase on a segment, we can free it and allocate memory for the next segment. As a result, the maximum memory usage is the sum of the checkpoints, since they are always in memory during the backward phase, and the maximum amount of memory of a segment since only one of them will appear in memory at any given time.

That is, two data d_i's from different segments will not be in memory simultaneously, and we can reduce the peak memory requirement. The pseudocode of the checkpoint algorithm is in Algorithm 2.

Algorithm 2 Neural Network Training with Checkpoint.

Require: Weights $w_1, \ldots, w_n$, input x (d_0), and the ground truth G.
Ensure: Updated $w_1, \ldots, w_n$.

```
for i ← 1 to n do                                          ▷ Forward Phase
    Allocate memory for d_i
    Compute d_i with d_{i−1} and w_i (Equation (1))
    if d_{i−1} is not a checkpoint then
        Free the memory of d_{i−1}
    end if
end for
for i ← n to 1 do                                          ▷ Backward Phase
    if d_i is a checkpoint then
        Free the memory of the segment after d_i
        Let d_h be the previous checkpoint, and allocate d_{h+1}, ..., d_{i−1}
        Recompute d_{h+1}, ..., d_{i−1} (Equation (1))
    end if
    Compute r_i (Equation (4))
    Compute g_i (Equation (5))
    Update w_i with g_i                                    ▷ Update the weight
end for
```

One immediate question one needs to answer is which subset from the data set $\{d_1, \ldots, d_n\}$ should be checkpoints. There is a trade-off between the amount of memory for checkpoints and the maximum-sized segment, i.e. the segment with the maximum sum of data memory. We can choose more checkpoints to reduce the memory of the maximum-sized segment, but the memory for checkpoints will increase. On the contrary, choosing fewer checkpoints will increase the memory of the maximum-sized segment. Therefore, it is essential to choose the correct checkpoints to balance the memory of the maximum-sized segment and the checkpoints.

2. Related work

In this section, we describe some works related to the checkpoint technique in the previous section for reducing the peak memory requirement.

In Deep Neural Networks training, the memory of data for backward propagation has outweighed that of model parameters [23]. This causes severe memory pressure in Algorithm 1. Chen et al. introduced checkpoints (Algorithm 2) to reduce the memory pressure by recomputing and gave a very simple estimate on the number of checkpoints required [2]. After that several efforts tried to find the optimal set of checkpoints to reduce the peak memory requirement [3].

2.1. Trading computation for memory

The question about the trade-off between allocating more memory for either checkpoints or the maximum-sized segment at the end of the previous section corresponds to the general methodology known as *trading computation for memory*. Chen et al. [2] showed that one can train a n-layer linear chain feedforward model in $O(\sqrt{n})$ memory at the cost of an additional forward pass in the backward phase during training. The memory cost to train the model is $O(n/\sqrt{n}) + O(\sqrt{n}) = O(\sqrt{n})$, which is optimal when the size of data of every layer is the same. In practice the $\sqrt{n}$ estimate might not be an optimal solution since the amount of data in different layers will differ. As a result, dividing the model into $\sqrt{n}$ number of equal-size segments does not guarantee the optimal results. Nevertheless, the simple technique still reduces the peak memory requirement.

Modern deep-learning platforms do support checkpoints. For example, PyTorch [18] provides the tool API `torch.utils.checkpoint_sequential` to implement Algorithm 2 for sequential models and `torch.utils.checkpoint` to specify segments by adding checkpoints. However, these functions are user-dependent, which means that its users have to find a set of checkpoints and divide their model into segments on their own accordingly to facilitate the tool API. To get the optimal result for all cases on linear models, we need to look for algorithms that work for models with non-uniform costs.

2.2. Finding checkpoints within a given budget automatically

To find the set of checkpoints, Chen et al. proposed another algorithm [2] in their work to generate a *memory allocation plan* (the placement of the checkpoints) within a given *budget* (the restriction on the memory of the maximum-sized segment). The algorithm can be seen as the *preprocessing step* [3] before the execution of Algorithm 2. In addition, it can be applied to general computation graphs without the assumption of uniform cost of layer data. In short, it works by running a heuristic search over all possible budgets, and for each given budget it does greedy allocation to get the exact memory for each memory allocation plan. Herrmann et al. [11] also proposed a method to find the checkpoints under a given memory budget but focused on minimizing the model training time. Their method is based on a dynamic programming algorithm of $O(mn^3)$ time complexity, where m is the memory limit and n is the number of layers. In contrast, this paper focuses on finding the optimal checkpoints with minimal peak memory usage, and we propose a linear time algorithm to solve the checkpoint selection problem.

2.3. Finding the optimal checkpoints for arbitrary DNN model

To find the optimal set of checkpoints to reduce peak memory, the scheme proposed by Feng et al. [3] is the first work that is applicable for arbitrary computation graphs (ACGs), as they did not pose any assumption on the computation graph of DNN models. They can achieve maximum memory cut-offs at the cost of moderate time overhead (~30%-50%).

To provide an overview of their algorithm, they denoted the computation graph of a DNN model as an acyclic-directed graph where the vertices are the intermediate tensors and the edges are the operations of the DNN model [3]. After the computation graph of a DNN model is built, they run their algorithm to divide the computation graph into *independent segments* (ISs), which means that there is no data dependency between the non-checkpoint vertexes in the subgraph and the other part of the graph. While the context is different, they use the same objective function for the peak memory requirement as the one defined by Chen et al., i.e. the memory of each segment is the sum of the memory of the non-checkpoint vertexes within the segment.

2.4. Summary

Table 1 summarizes checkpoint models and algorithms for linear neural networks. The upper half of Table 1 focuses on the memory usage model proposed by Chen et al. [2]. In this paper, we improve the execution time for finding the minimum memory usage from $O(n^3 \log n)$ to $O(n^3)$.

The lower half of Table 1 focuses on the memory usage model derived from PyTorch's implementation. The memory model of Algorithm 2 is inconsistent with that of PyTorch's implementation. In this paper, we derive the memory usage model that fits the memory usage reported by PyTorch, as will be demonstrated in Section 5. Based on this accurate memory usage model, we design a linear time algorithm that ensures optimal memory utilization (Section 4). Note that this paper focuses on finding the minimum memory footprint and does not consider the trade-off between memory and computation time.

Table 1
The comparison of optimization on linear neural networks.

model / works	[2]	[3]	[11]	this paper
checkpoint	✓	✓	✓	✓
layers of different sizes		✓	✓	✓
memory model of Algorithm 2	✓	✓		✓
the execution time of Algorithm 2		$O(n^3 \log n)$		$O(n^3)$
consider memory usage optimization (e.g. PyTorch)			✓	✓
consider output gradient memory in PyTorch			✓	✓
trade-off between memory and execution time			✓	
find the minimum memory usage				✓

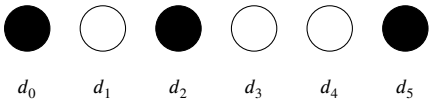


Fig. 1. An example of three checkpoints (in black) and two segments: {d1} and {d3, d4} (in white).

3. Checkpoint selection problem

3.1. Problem definition

We consider a sequence of $n+1$ data $(d_0, \ldots, d_n)$, where d_0 is the input and d_n is the output, as in Section 1. For ease of explanation, d_i denotes both the data and the size of the data. Since d_0 is the input and it needs to be in memory, we choose it as a checkpoint. In addition, the training generates the output d_n after the forward phase and immediately uses it at the beginning of the backward phase, so we cannot save any memory by *not* choosing it as a checkpoint. As a result, without loss of generality, we assume both d_0 and d_n are checkpoints in this paper.

Now we choose m additional checkpoints to save memory by recomputing non-checkpoints during the backward phase, as in Algorithm 2. Recall that the data between two checkpoints becomes a segment. Now the set C of $m+2$ checkpoints (including d_0 and d_n) will partition the sequence $(d_0, \ldots, d_n)$ into $(s_1, \ldots, s_{m+1})$ segments, where each segment is the set of data between two checkpoints. Note that two checkpoints may be adjacent so that a segment may be empty. Fig. 1 shows the partitioning of a model into three checkpoints and two segments.

From the previous discussion, we consider an objective function consisting of the *checkpoint memory* and the *segment memory*. The *checkpoint memory* is the sum of the memory of all checkpoints, i.e., $\sum_{i \in C} d_i$. The *segment memory* is the maximum of the sum of memory in every segment, i.e., $\max_i \sum_{j \in s_i} d_j$. The goal is to find a checkpoint subset C which minimizes the sum of checkpoint memory and segment memory (Equation (6)). We will denote this problem as the *checkpoint selection problem*.

$$\sum_{i \in C} d_i + \max_{1 \le i \le m+1} \left(\sum_{j \in s_i} d_j \right) \tag{6}$$

3.2. Dynamic programming

In this section, we solve the checkpoint selection problem by an algorithm based on dynamic programming.

The algorithm consists of two steps:

1. For each possible segment memory s, find the minimal checkpoint memory c_s such that the segment memory does not exceed s.
2. Find the minimum of $s + c_s$.

The implementation of step 2 is a simple algorithm iterating over all possible $s + c_s$. In the next paragraph, we will discuss how to find c_s for all s in step 1.

Step 1 of the algorithm is implemented using dynamic programming as follows. Define a function $M(i, s)$ as the minimum checkpoint memory for the sequence $(d_1, \ldots, d_i)$ such that the segment memory does not exceed s. Here c_s equals $M(n, s)$. Now we derive the recursion for M. Each $M(i, s)$ is the sum of the checkpoint d_j, closest to d_i, and the minimum checkpoint memory between $(d_0, \ldots, d_{j-1})$. Since the segment memory does not exceed s, the term j must satisfy Inequality (7).

$$\sum_{k=j+1}^{i} d_k \le s \tag{7}$$

Therefore, the term $M(i, s)$ is the minimum of $d_j + M(j-1, s)$ for all j satisfying Equation (7). To express the recursion as a formula, let $l(i)$ be the minimum j that satisfies Inequality (7). The recursion for M is given in Equation (8).

$$M(i, s) = \min_{l(i) \le j \le i} (d_j + M(j-1, s)) \tag{8}$$

Algorithm 3 Checkpoint Selection.

Require: data sizes $d_0, \ldots, d_n$.
Ensure: The minimum cost of checkpoint selection
  for all segment cost s **do**
    Compute each $l(i)$ for $1 \le i \le n$
    $M(0, s) = d_0$
    $Q = \{0\}$
    for $i \leftarrow 1$ to n **do**
      Remove those q_h from the head of Q such that $l(i) > q_h$.
      Set $M(i, s)$ to be $d_q + M(q-1, s)$, where q is the head of Q.
      Remove those q_t from the tail of Q such that $d_{q_t} + M(q_t - 1, s) > d_i + M(i-1, s)$.
      Add i to the tail of Q.
    end for
  end for
  return $\min_s (s + M(n, s))$ ▷ the answer

Algorithm 3 is the implementation details of the algorithm that solves the checkpoint selection problem. We claim that, in order to prove the correctness of our algorithm,

- the queue Q contains the j's achieving the minimum in Equation (8), and
- the head of Q is $\operatorname{argmin}_{l(i) \le j \le i} (d_j + M(j-1, s))$.

The latter is valid, since in the i-th iteration, we remove those j from the tail of Q such that $d_i + M(i-1, s) < d_j + M(j-1, s)$. Therefore, the head of Q, say q, is the element with the smallest $d_q + M(q-1, s)$ in Q.

To prove the former, we ensure that in the i-th iteration, all j's with $j < i$ have entered Q, and only the ones that do not achieve the minimum in Equation (8) are removed from Q. All j's with $j < i$ have entered Q since we add each j to the tail of Q before the end of the j-th iteration. If $u < i$ is removed from Q because some $v < i$ induces $u < l(v)$, then since l is monotonically increasing, we have $l(v) < l(i)$, implying that $u < l(i)$. Therefore, the term u is not a valid j in Equation (8). If $u < i$ is removed from Q because some $v < i$ induces $d_u + d(u-1, s) > d_v + d(v-1, s)$, then whenever u is a valid j in Equation (8), we can always find another j,

namely v, with a smaller $d_j + M(j-1,s)$. Therefore, the term u is not the j achieving the minimum in Equation (8).

We analyze the time complexity of Algorithm 3 in the following paragraphs. To analyze the time complexity of completing the recursion $M(i,s)$ for a fixed s, three operations are put into consideration: computing each $l(i)$ with $1 \le i \le n$, assigning values to each $M(i,s)$, and maintaining the queue Q.

- Computing each $l(i)$ takes $O(n)$ time by maintaining the sliding window that starts at $l(i)$ and ends at i over the sequence $(1,2,\dots,n)$. This is because each element in the sequence $(0,1,\dots,n)$ enters and exits the window only once, and each insertion and deletion takes $O(1)$ time.
- In the i-th iteration for each segment memory usage s, it takes $O(1)$ time to find the head of Q, say q, and assign $d_q + M(q-1,s)$ to each $M(i,s)$. Therefore, it takes $O(n)$ time to assign values to each $M(i,s)$ for a fixed s.
- For each segment memory usage s, each number from 0 to n enters and leaves the queue Q once, resulting in $O(n)$ insertions and deletions. Since each insertion and deletion occurs only at the head and tail of Q, both operations have $O(1)$ time complexity. Therefore, the maintenance of Q takes $O(n)$ time.

As a result, it takes $O(n)$ time to complete the recursion of $M(i,s)$ for a fixed s.

Next, we find the number of different segment memory usages. We cannot iterate each value between 1 and $\sum_{i=1}^{n} d_i$ since it leads to a pseudo-polynomial time algorithm. In fact, there are $O(n^2)$ possible segment memory usages, obtained by enumerating the start and end points of the segments.

As a result, there are $O(n^2)$ different segment memory usages, each taking $O(n)$ time to compute. Therefore, we obtain the following theorem.

Theorem 1. *The checkpoint selection problem is $O(n^3)$-time solvable, where n is the number of neural network layers.*

4. PyTorch implementation

We use Algorithm 2 to estimate the memory usage and compare it with the result from PyTorch, a state-of-the-art deep learning platform that supports checkpoints. However, the memory usage reported by PyTorch does not match the simulated results from Algorithm 2. After tracing the implementation of PyTorch, we realized the following.

1. First, PyTorch does *not* keep all checkpoints in memory all the time, as Equation (6) indicates. We observe that PyTorch released the memory of d_i (checkpoint or not) when it is no longer used. That is, it does not release the memory in batch, as Algorithm 2 suggests.
2. Algorithm 2 does not consider the memory of r_i (Equation (4)). The size of this *output gradient* is the same as d_i, which is quite significant. In addition, PyTorch will allocate a data buffer to fit the maximum output gradient d_i within a segment. As a result, the size of maximum d_i should be counted *twice* for an accurate estimate of the memory requirement of PyTorch.

We now describe a more memory-saving checkpoint mechanism. We conjectured that PyTorch uses this mechanism, instead of Algorithm 2, and we verified this conjecture by experiments in Section 5. Therefore, this is a more accurate description of the PyTorch implementation. The pseudocode of this memory management algorithm is in Algorithm 4.

The new mechanism will release the memory of d_i as soon as we do not need them. The forward phase is the same as the previous mechanism; we only keep checkpoints in memory. During the backward phase, we will have two cases for the index i in Algorithm 4.

Algorithm 4 Neural Network Training with Checkpoints as Done by PyTorch.

Require: Weights $w_1,\dots,w_n$, input x (d_0), and the ground truth G.
Ensure: Updated $w_1,\dots,w_n$.

for $i \leftarrow 1$ to n **do** ▷ Forward Phase
  Allocate memory for d_i
  Compute d_i with d_{i-1} and w_i
  if d_{i-1} is not a checkpoint **then**
    Free the memory of d_{i-1}
  end if
end for
for $i \leftarrow n$ to 1 **do** ▷ Backward Phase
  if d_i is a checkpoint **then**
    Let d_h be the previous checkpoint. Allocate and recompute $d_{h+1},\dots,d_{i-1}$
    Release the buffer for the previous output gradients
    Allocate a new buffer r of size $\max(d_h,\dots,d_{i-1})$ ▷ output gradient
  end if
  Compute r_i (Equation (4))
  Compute g_i (Equation (5))
  Update w_i with g_i ▷ Update the weight
  Release the memory d_i ▷ as PyTorch implementation
end for

- If d_i is a checkpoint, then we will first find the previous checkpoint d_h and recompute all data in the segment between d_{h+1} and d_{i-1}, where d_h is the previous checkpoint.
- We then free the memory of the output gradient of the previous segment.
- We then allocate a new buffer for the output gradient of this segment, of size $\max(d_h,\dots,d_{i-1})$.
- We then update the weight w_i as in Algorithm 2. Finally, just as in the PyTorch implementation, we free the memory of d_i since we no longer need it.

With these observations, we derive a more accurate memory consumption model and verify it with experiments in Section 5.2. Now we focus on the theoretical aspects of finding the set of checkpoints to minimize memory use for this new model.

One can think of this new checkpoint mechanism as it shrinks the current segment until it reaches the next checkpoint, and then it recomputes the next segment and starts shrinking it. Also, it always keeps a buffer of the size of the maximum among the current segment and its left checkpoint as the output gradient. Let C be the set of checkpoints, $d_i \in C$, and d_h is the checkpoint before d_i in C. The memory usage for the i-th backward phase $m(i)$ is in Equation (9), where $C(i)$ is the set of checkpoints with an index no larger than i. Note that we only need to define $m(i)$ for those i's where $d_i \in C$ to find the maximum memory usage since the memory usage from the i-th iteration to the $h-1$-th iteration always decreases, since Algorithm 4 always free memory at the end of the iteration. For example, if $d_{i'}$ is a data between d_h and d_i, i.e., $h < i' < i$, then memory usage $m(i')$ is in Equation (10). Note that PyTorch implementation (Algorithm 4) still maintains the output gradient buffer of size $\max_{h \le k < i}(d_k)$ when the backward phase goes to $d_{i'}$. It is obvious that $m(i') < m(i)$, so we only need to define $m(i)$ for those i's where $d_i \in C$ to find the maximum.

$$m(i) = \sum_{d \in C(i)} d + \sum_{k=h+1}^{i-1} d_k + \max_{h \le k < i}(d_k) \tag{9}$$

$$m(i') = \sum_{d \in C(h)} d + \sum_{k=h+1}^{i'} d_k + \max_{h \le k < i}(d_k) \tag{10}$$

For ease of notation, we define $s(h,i)$ as in Equation (11), so we can rewrite Equation (9) into (12).

$$s(h,i) = \sum_{k=h+1}^{i-1} d_k + \max_{h \le k < i}(d_k) \tag{11}$$

$$m(i) = \sum_{d \in C(i)} d + s(h,i) \tag{12}$$

Given the d_i for $1 \le i \le n$, the goal is to find the set of checkpoints C which minimizes the maximum among all i for Equation (12), for all d_i in C. We will use *dynamic checkpoint selection problem* to denote this optimization problem.

4.1. Dynamic programming

We again use dynamic programming to solve the dynamic checkpoint selection problem.

Define a function $M(i)$, representing the minimum memory usage from layer i to layer n, given that layer i is the checkpoint. Here, the term $M(1)$ is the answer to the dynamic checkpoint selection problem. Now we derive the recursion of M. For each term $M(i)$, $1 \le i < n$, consider the next checkpoint: d_j, $i < j < n$. There are two possibilities for minimum memory usage:

- The memory usage peaks when the backpropagation updates the weights between layer j and layer n. Therefore, the memory usage is $d_i + M(j)$.
- The memory usage peaks when the backpropagation updates the weights between layer i and layer j. Therefore, the memory usage is $d_i + s(i,j) + d_j$.

As a result, we derive the recursion of M as in Equation (13).

$$M(i) = \min_{i<j<n}(\max(d_i + M(j), d_i + s(i,j) + d_j)) \tag{13}$$

Here we discuss the time complexity of this recursion. To find a single $M(i)$ using Equation (13), we enumerate all j between i and n and then find the one that minimizes the larger values between $d_i + M(j)$ and $d_i + s(i,j) + d_j$. A simple algorithm takes $O(n)$ time to compute each $M(i)$, and there are n such $M(i)$ values to compute. Therefore, the time for computing all $M(i)$'s is $O(n^2)$, if we precompute all $s(i,j)$, which also takes $O(n^2)$ time. See Algorithm 5 for the implementation details.

Algorithm 5 Dynamic Checkpoint Selection in $O(n^2)$ time.

Require: data sizes $d_0, \ldots, d_n$.
Ensure: The minimum cost of checkpoint selection

```
M(n) = d_n
for i ← n − 1 to 1 do
    M(i) = ∞
    for j ← i + 1 to n do
        M(i) = min(M(i), max(d_i + M(j), d_i + s(i, j) + d_j))
    end for
end for
return M(1)                                   ▷ the answer
```

Theorem 2. *The dynamic checkpoint selection problem is $O(n^2)$-time solvable, where n is the number of neural network layers.*

4.2. Linear time

We now describe how to compute Equation (13) in linear time of n. First, we rewrite Equation (13) into Equation (14).

$$M(i) = d_i + \min_{i<j<n}(\max(M(j), s(i,j) + d_j)) \tag{14}$$

To ease the notation we define $U(i,j)$ as in Equation (15) and simplify Equation (14) into (16).

$$U(i,j) = s(i,j) + d_j \tag{15}$$

$$M(i) = d_i + \min_{i<j<n}(\max(M(j), U(i,j))) \tag{16}$$

The linear-time algorithm is derived from two observations:

- The computation of $M(i)$ using Equation (16) can be reduced to finding the minimum of the larger values between an increasing function and a decreasing queue.
- The index that yields the minimum in Equation (16) when computing $M(i)$ is no greater than the index that yields the minimum in Equation (16) when computing $M(i+1)$.

4.2.1. The first observation

The function $U(i,j)$ is monotonic of j for a given i.

Lemma 1. *$U(i,j)$ is an increasing function of j for a given i.*

Proof.

$$U(i,j+1) \tag{17}$$

$$= s(i,j+1) + d_{j+1} \tag{18}$$

$$= \Big(\sum_{k=i+1}^{j} d_k + \max_{i \le k \le j}(d_k)\Big) + d_{j+1} \tag{19}$$

$$> d_j + \sum_{k=i+1}^{j-1} d_k + \max_{i \le k \le j}(d_k) \tag{20}$$

$$> \Big(\sum_{k=i+1}^{j-1} d_k + \max_{i \le k < j}(d_k)\Big) + d_j \tag{21}$$

$$= U(i,j) \quad \square \tag{22}$$

We use a monotonic queue Q to store a sorted subset of $M(i)$'s, for i between 1 and n. We compute $M(i)$ for i from n down to 1. After computing $M(i)$, we remove those M's from the head of Q that are no less than $M(i)$, then add $M(i)$ to the head of Q. As a result, we acquire Lemma 2.

Lemma 2. *Q is a monotonically decreasing queue with respect to the index of function M.*

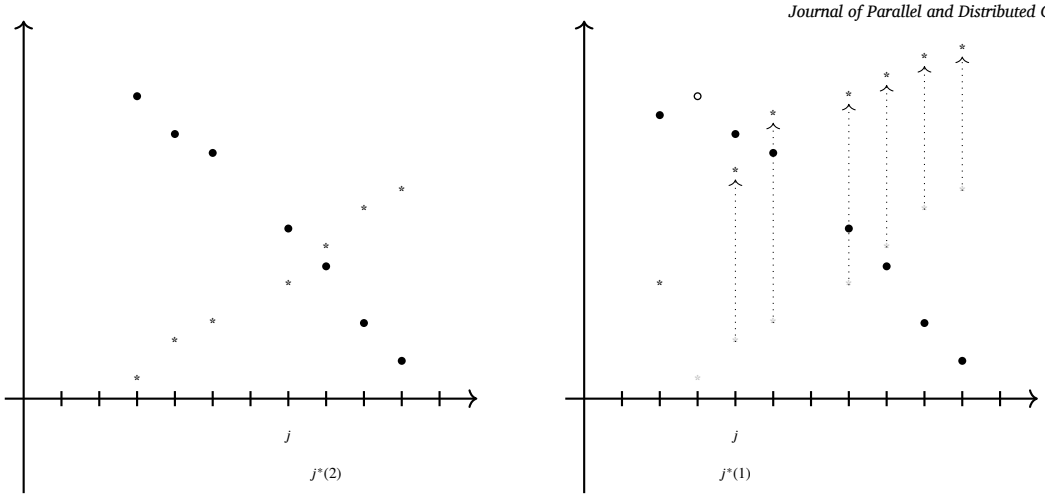
Proof. The lemma follows from the construction of Q. $\square$

For ease of notation, we use Q_i to denote the monotonic queue Q after we added $M(i)$ to it. We observe that only those $M(j)$'s in Q need to be considered when applying Equation (16) to compute $M(i)$. Formally we have Equation (23).

$$M(i) = d_i + \min_{j \in Q_{i-1}}(\max(M(j), U(i,j))) = d_i + \min_{i<j<n}(\max(M(j), U(i,j))) \tag{23}$$

Lemma 3. *When we compute $M(i)$ with Equation (16), we only need to consider those indices j's in Q_{i-1}, i.e., we have Equation (23).*

Proof. Consider $M(a)$ that removes $M(b)$ during the construction of Q. By the construction of Q, we have $M(a) \le M(b)$. In addition, we have $a < b$ because we compute $M(i)$ for i from n to 1. This implies that $U(i,a) < U(i,b)$ since Lemma 1 states that $U(i,j)$ is increasing for a fixed i. The preceding two inequalities, $M(a) \le M(b)$ and $U(i,a) < U(i,b)$, imply $\max(M(a), U(i,a)) \le \max(M(b), U(i,b))$, which indicates that we can safely ignore the case $j = b$ when computing $M(i)$ using Equation (16). Therefore, the Lemma follows and Equation (23) is valid. $\square$



(a) The values of $M(\cdot)$ and $U(2,\cdot)$ when we compute $M(2)$.

(b) The values of $M(\cdot)$ and $U(1,\cdot)$ when we compute $M(1)$. The gray stars are the values of $U(2,\cdot)$. The hollow dot represents the element at the head of Q larger than $M(2)$ and is therefore removed from Q before $M(2)$ is inserted.

Fig. 2. The values of $M(\cdot)$ and $U(i,\cdot)$ when we compute $M(2)$ and $M(1)$. The x-coordinate represents the index of $M(\cdot)$ and $U(i,\cdot)$, while the y-coordinate represents their corresponding values. Black dots indicate the values of $M(\cdot)$ present in the queue Q_i, and stars represent the values of $U(i,\cdot)$. The x-coordinate i without a black dot indicates that $M(i)$ has been removed from the queue Q. The j below the x-axis marks the value of j after the "while" loop in Algorithm 6. The $j^*(i)$ below the x-axis marks the value of updated $j^*_(i)$.

Combining Lemma 1, 2, and 3, we obtain the first observation of our linear time algorithm: the value of $M(i)$ is the sum of d_i and the minimum of the larger value between an increasing function $U(i,\cdot)$ and a decreasing queue Q with respect to j, the index of M's in Q.

To simplify the description, we use $j^*(i)$ to denote the j that minimizes Equation (16), as defined in Equation (24). This observation reduces the search for $j^*(i)$ in Equation (16) from considering each $U(i,j)$ and $M(j)$ for all j between i and n to identifying the intersection of the increasing function U and the decreasing queue Q if they intersect. If the function values of U and the values in Q do not intersect, then the $j^*(i)$ will be n or i. Fig. 2a illustrates this observation.

$$j^*(i) = \arg\min_{i<j<n}(\max(M(j), U(i,j))) \tag{24}$$

4.2.2. The second observation

We observe how $U(i,j)$ changes when i decreases for a fixed j.

$$U(i-1,j) \tag{25}$$
$$= s(i-1,j) + d_j$$
$$= \sum_{k=i}^{j-1} d_k + \max_{i-1\le k<j}(d_k) + d_j \tag{26}$$
$$= (d_i + \sum_{k=i+1}^{j-1} d_k) + (\max(d_{i-1}, \max_{i\le k<j}(d_k))) + d_j$$
$$> \sum_{k=i+1}^{j-1} d_k + \max_{i\le k<j}(d_k) + d_j \tag{27}$$
$$= s(i,j) + d_j \tag{28}$$
$$= U(i,j) \tag{29}$$

The inequality $U(i-1,j) > U(i,j)$ indicates that after we compute $M(i)$ by finding $j^*(i)$ that minimizes Equation (16), we only need to consider those j's that are no greater than $j^*(i)$ when we compute $M(i-1)$ through minimizing Equation (16). The dotted arrows in Fig. 2b illustrate the change from $U(2,j)$ to $U(1,j)$.

Lemma 4. *j^* is a increasing function of i, i.e., $j^*(i-1) \le j^*(i)$.*

Proof. In Equation (16), $M(j)$ is a function of j only and not a function of i. However, for all j from $i+1$ to $n-1$, $U(i-1,j) > U(i,j)$, and $U(i-1,j)$ is a increasing function of j. As a result, if a $M(j) \in Q$ and $U(i-1,j)$ intersect, they will intersect at a j value that is no greater than $j^*(i)$. □

Lemma 4 leads to the second observation: to find the $j^*(i-1)$, the index that minimizes $M(i-1)$ in Equation (16), we only need to consider those j's that are no greater than $j^*(i)$. In other words, when computing all $M(i)$ for i from n to 1, we only need to find the j that minimizes $M(i)$ in the direction of decreasing j.

Fig. 2 illustrates the second observation. According to the first observation, the minimum is yielded at the intersection of the black stars and dots. With the transition from $U(2,\cdot)$ to $U(1,\cdot)$, as indicated by the dotted arrows in Fig. 2b, we observe that the index of the intersection between the black stars and dots when computing $M(2)$ is no less than the index of their intersection when computing $M(1)$. Therefore, we have $j^*(1) \le j^*(2)$.

4.2.3. The linear time algorithm

We list the pseudo-code of the dynamic checkpoint selection problem in Algorithm 6.

Theorem 3. *The dynamic checkpoint selection problem is $O(n)$-time solvable, where n is the number of neural network layers.*

Proof. To prove the correctness, we show that during the i-th iteration, Algorithm 6 obtains the correct $M(i)$. According to Lemma 4 in the second observation, the search for the $j^*(i)$ begins at $j^*(i+1)$, which is implemented in line 4. The first observation indicates that $j^*(i)$ occurs at the intersection of $U(i,\cdot)$ and Q. The "while" loop from line 6 to line 12, computing the first index j before this intersection, implements this observation. This is achieved since the function $U(i,\cdot)$ and the maintenance of Q are the same as described in the first observation. Subsequently, we assign $M(i)$ as the smaller value between $d_i + \max(M(j), U(i,j))$ and $d_i + \max(M(j+1), U(i,j+1))$ as indicated by Equation (16). The two subfigures of Fig. 2 illustrate why both j, the first index before the intersection, and $j+1$, the first index after the intersection, may be the index

Algorithm 6 Dynamic Checkpoint Selection Problem in $O(n)$ time.

Require: data sizes $d_1, \ldots, d_n$.
Ensure: The minimum cost of the dynamic checkpoint selection problem.
1: $Q = \emptyset$
2: $j^* = n$
3: **for** $i \leftarrow n$ to 1 **do** ▷ Compute $M(i)$ for i from n to 1
4: $j = j^*$
5: **if** $i < n$ **then**
6: Compute $U(i, j)$ from $U(i+1, j)$ using Equation (26)
7: **end if**
8: **while** $U(i, j) \geq M(j)$ and $M(j)$ is not the first element in Q **do** ▷ repeat until a crossover
9: Set j' to be the index of the previous element of $M(j)$ in Q
10: Compute $U(i, j')$ from $U(i, j)$ using Equation (19)
11: $j = j'$
12: **end while**
13: Set $M(i)$ to $d_i + \min(\max(M(j), U(i, j)), \max(M(j+1), U(i, j+1)))$
14: Set j^* to j or $j+1$ according to the minimum from the previous statement.
15: Remove those M's at the beginning of Q that are no less than $M(i)$
16: Add $M(i)$ at the beginning of Q
17: **end for**
18: return $M(1)$ ▷ the answer

$j^*(i)$ that minimizes Equation (16). We have thus proven the correctness of the Algorithm 6.

We will now prove that the time complexity of Algorithm 6 is $O(n)$. The time complexity of Algorithm 6 consists of the time to maintain Q and to compute each $U(i, j)$ in line 6 and line 10.

Each $M(i)$ will enter and exit Q once, from either its head or its tail. Therefore, the time complexity of maintaining Q is $O(n)$.

The computation of $U(i, j)$ at line 6 is performed $n-1$ times. Each $U(i, j)$ is derived from $U(i+1, j)$ using Equation (26). By maintaining $\sum_{k=i+2}^{j-1} d_k$ and $\max_{i+1 \leq k < j}(d_k)$, the terms $\sum_{k=i+1}^{j-1} d_k$ and $\max_{i \leq k < j}(d_k)$ in Equation (26) can be computed in $O(1)$ time. As a result, the computation of $U(i, j)$ at line 6 across all iterations takes O(n) time.

Each computation of $U(i, j')$ at line 10 takes $O(j - j')$ time by tracking $\sum_{k=i+1}^{j} d_k$ and $\max_{i \leq k \leq j}(d_k)$ in Equation (19). Moreover, after the computation, the value of j is decreased by $j - j'$ (line 11). The time complexity of the computation in line 10, therefore, depends on the range by which j is reduced. Initially, j equals n. It remains positive throughout the computation, and in each iteration of the "for" loop, the value of j can increase by at most 1 (line 14). Therefore, the range by which j is reduced is bounded by $2n$, implying the $O(n)$ time complexity of the computation in line 10.

As a result, the time complexity of Algorithm 6 is $O(n)$. □

5. Experiment

In this section, we evaluate our algorithms presented in the previous sections. We profile the GPU memory usage when training two linear feedforward DNN models with the optimal checkpoint subset returned by the algorithms. We do the same for comparison by profiling the training of the same models with non-optimal checkpoint subsets. Finally, we run the algorithm implementation of Feng et al. [3] to find and compare the differences between their works and ours.

5.1. Environment settings

5.1.1. Models and definitions

We use VGG-19 [22] and AlexNet [16] as our DNN model to evaluate our algorithm. VGG-19 has 16 convolution layers, 5 max-pooling layers, and 3 fully connected layers. AlexNet has 5 convolution layers, 3 max-pooling layers, and 3 fully connected layers.

We define the term *training phase index* to describe every stage of training, including both the forward and backward phases. It is a number ranging from 0 to $2N+1$, where N is the number of layers of the model. The value 0 represents the stage before the training, and $2N+1$ represents the stage after training. The training is in the forward phase when i is in the range $1 \leq i \leq N$, and it is in the backward phase when i is in the range $N+1 \leq i \leq 2N$. Now we can index the entire training. Notice that we always measure the GPU memory at the end of every stage. For ease of index calculation, we do not merge the forward phase and the backward phase of the last layer. Figs. 3 and 4 depict the training phase indices for VGG-19 and AlexNet, respectively. For example, the training phase index 29 of VGG-19 means that it is the stage in the backward phase of the 20th layer, i.e. we are in the backward phase of the last convolutional layer with 512 output channels.

5.1.2. Benchmarks

We use ImageNet [21] to evaluate our algorithms. ImageNet consists of around 1.3 million color images of different sizes in 1000 classes. We crop all images into size 224 by 224 before feeding them into the model.

5.1.3. Implementation

We use PyTorch [18] deep learning platform to implement the two DNN models VGG-19 and AlexNet. To implement the recomputing of a segment in the checkpoint technique, we use the PyTorch API `torch.utils.checkpoint`. To profile the GPU memory usage, we use the PyTorch API `torch.cuda.memory_allocated`. We include the fully-connected layers of both VGG-19 and AlexNet when selecting the candidate of checkpoints, in addition to convolution layers. This is because the memory cost of fully-connected layers is large. Additionally, we exclude the layers of AdaptiveAvgPool2d, Flatten, and Dropout in VGG-19, but include these layers as checkpoint candidates in AlexNet. This simplifies the selection of checkpoints for the $O(\sqrt{n})$ algorithm [2]. We perform all experiments with GPUs of the same specs, NVIDIA GeForce RTX 3090, 24 GiB memory on a server with 16-core, 2.90 GHz Intel Xeon Gold 6226R CPU, 192 GiB RAM.

5.2. Algorithm prediction versus PyTorch report

In this experiment, we demonstrate the importance of the consideration of the output gradient buffer as we have mentioned in section 4. We show that the prediction of GPU memory usage by our algorithm is aligned with the report from PyTorch.

In Fig. 5, we profile our training of VGG-19 with the checkpoint subset returned by our algorithm and draw the GPU memory usage prediction using the same checkpoint subset. The blue dashed line represents the GPU memory usage reported by PyTorch. On the other hand, the red line is the GPU memory usage predicted by our algorithm. Notice that we use the checkpoint subset {3, 11, 24} reported by our algorithm when running on VGG-19 to mark the corresponding stages on the x-axis for both the forward and backward phases, so the six ticks on the x-axis are all stages about checkpoints. The average error of our prediction is 2.8%.

5.3. Comparison with $O(\sqrt{n})$ memory cost algorithm

In this experiment, we compare the GPU memory usage when training VGG-19 using the checkpoint subset provided by our algorithm and the one by the sublinear memory cost algorithm proposed by Chen et al.

In Fig. 6, the red line represents the GPU memory usage of the training on VGG-19 reported by PyTorch using the checkpoint subset returned by our algorithm. On the other hand, the blue dashed line represents the GPU memory usage of the training using the checkpoint subset calculated by the sublinear memory cost algorithm. We mark the ticks of the x-axis using the checkpoint subset of the latter, which is the set {5, 10, 15, 20, 24}.

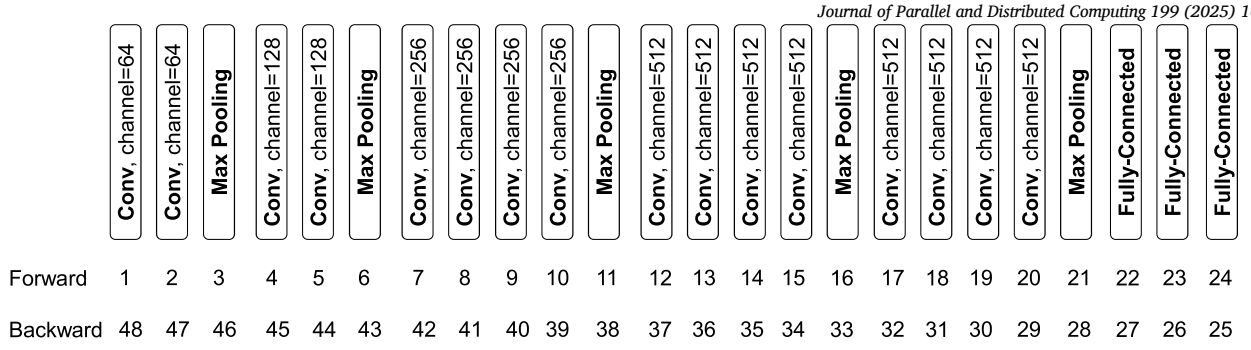


Fig. 3. Training phase indices of VGG-19.

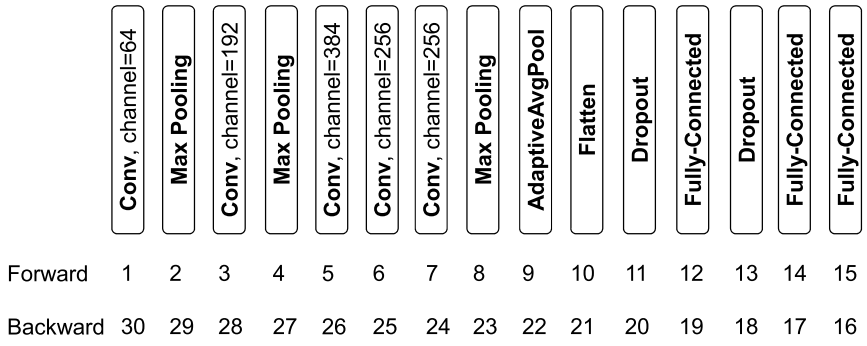


Fig. 4. Training phase indices of AlexNet.

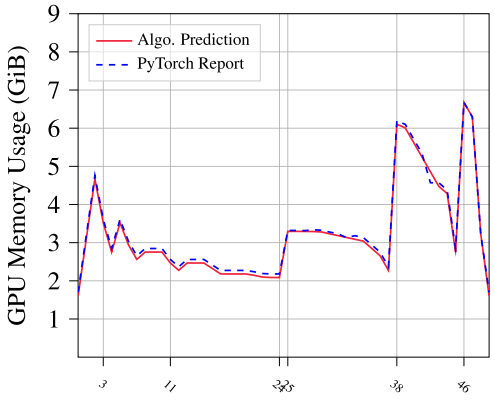


Fig. 5. GPU Memory Usage: Algorithm Prediction vs PyTorch Report on VGG-19 with Checkpoint Subset {3, 11, 24}.

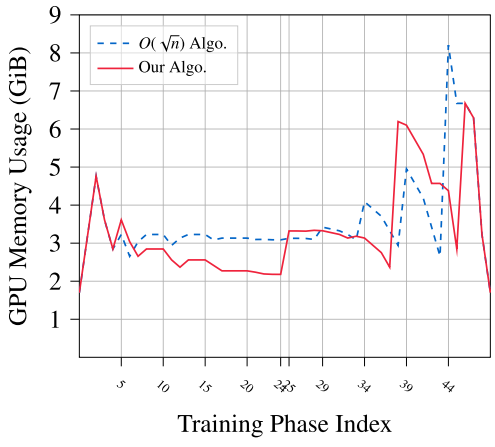


Fig. 6. GPU Memory Usage: Our Algorithm vs $O(\sqrt{n})$ Memory cost Algorithm Proposed by Chen et al. on VGG-19.

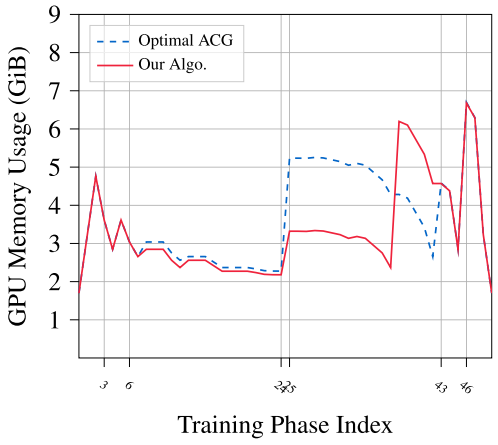


Fig. 7. GPU Memory Usage: Our Algorithm vs Optimal Arbitrary Computation Graph (ACG) Algorithm Proposed by Feng et al. on VGG-19.

5.4. Comparison with optimal arbitrary computation graph algorithm

In this experiment, we compare the GPU memory usage when training VGG-19 using the checkpoint subset provided by our algorithm and the one by the optimal arbitrary computation graph algorithm proposed by Feng et al.

In Fig. 7, the red line represents the GPU memory usage of the training on VGG-19 reported by PyTorch using the checkpoint subset returned by our algorithm. On the other hand, the blue dashed line represents the GPU memory usage of the training using the checkpoint subset returned by the optimal arbitrary computation graph algorithm. We mark the ticks of the x-axis using the checkpoint subset of the latter, which is the set {3, 6, 24}. In the figure, we can see that both algorithms have the same peak memory usage that occurs at the start of the last segment in the backward phase. During the same training phase index range from 25 to 43, the training using the optimal ACG algorithm spends around 2 GiB more memory compared to our algorithm on the

Table 2
The Profiling Results of Peak Memory Usage: Finding the Checkpoint Subset Using Different Methods.

Peak Mem.(MiB)	Algorithm 6	Algorithm 3	ACG Solver	$O(\sqrt{n})$	PyTorch
VGG-19, b = 128	6,444	6,835	6,836	8,404	11,262
	{2,4,6,9,11,14,16,19,21,23,24}	{3,6,24}	{3,6,24}	{5,10,15,20,24}	
VGG-19, b = 256	11,220	12,004	12,004	15,139	20,850
VGG-19, b = 320	13,609	14,592	14,592	18,511	OOM
VGG-19, b = 384	15,997	17,177	17,177	OOM	OOM
VGG-19, b = 400	16,594	17,824	17,824	OOM	OOM
AlexNet, b = 128	1,002	1,003	1,003	1,106	1,174
	{2,4,6,8,12,14,15}	{2,4,12,15}	{2,4,12,15}	{4,8,12,15}	
AlexNet, b = 4096	9,866	9,866	9,866	13,182	15,275

first half and more of the first segment during the backward phase. On the other hand, our algorithm also spends around 2 GiB more memory compared to the optimal ACG algorithm on the remaining part of the first segment during the backward phase.

5.5. Comparison: the peak memory usage on different experiment settings

In this experiment, we compare the GPU memory usage when training both VGG-19 and AlexNet with different batch sizes and checkpoint subsets that are computed using different algorithms. The results are shown in Table 2. In the first column, the shorthand b = N means that we use the number N as the batch size for all experiments of the current row. The abbreviation OOM means that the experiment cannot be carried out due to the out-of-memory error reported by PyTorch. For the benchmark, we use a single batch from the ImageNet database for testing. (In the implementation of Feng et al., they randomly generate a batch of images for testing.)

For the VGG-19 results in Table 2, both Algorithm 3 and the optimal ACG algorithm have the same peak memory usage reported by PyTorch since the checkpoint subsets returned by the two algorithms are the same. On the other hand, Algorithm 6 has the lowest peak GPU memory usage. The checkpoint subset returned by Algorithm 3 and the optimal ACG algorithm is {3, 6, 24}. The checkpoint subset returned by Algorithm 6 is {2, 4, 6, 9, 11, 14, 16, 19, 21, 23, 24}. Compared to the $O(\sqrt{n})$ algorithm and ACG Solver, Algorithm 6 reduces 2 GiB (23%) and 392 MiB (5.7%) of peak memory usage with a batch size of 128, respectively. More importantly, the checkpoint subsets do not change when we increase the batch size since the batch size is a factor in the output data size of every layer.

5.6. The granularity of model specs matters: testing the optimal checkpoints on AlexNet with extended model specs

In this subsection, we run Algorithm 6 on AlexNet model to get the optimal checkpoint subset, and train and profile AlexNet with these checkpoints accordingly. From our prior experiments with VGG-19 models, we observe that the peak GPU memory usage usually occurs at one of the max-pooling layers. But in the plain model specs of AlexNet, all of its max-pooling layers are embedded into *larger, abstract* convolution layers, i.e. it is *hidden* from the selection of checkpoints. We move all of these max-pooling layers out of the convolution layers since the behavior of our algorithm is dependent on the granularity, i.e. the length, of the model specs. We expect that the peak GPU memory usage will decrease with the new model specs because more combinations are considered.

5.6.1. AlexNet: using the plain specs

In this trial, we use the plain model specs of AlexNet, i.e. it has 5 convolution layers and 3 fully-connected layers. The result in Fig. 8 indicates that our algorithm chose checkpoints {3, 4, 5, 9, 11, 12}. This means that the third and fourth convolution layers (Conv2d/ReLU), the fifth convolution layer (Conv2d/ReLU + MaxPool2d), and all three fully-connected layers are chosen as checkpoints.

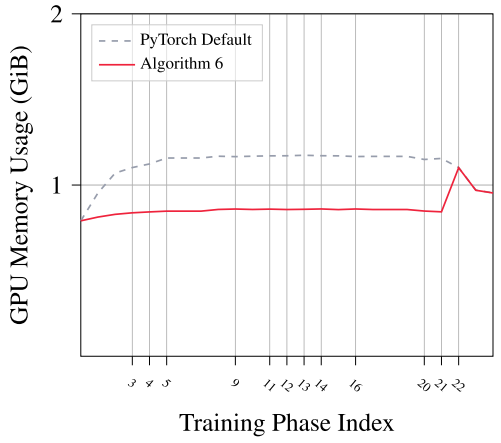


Fig. 8. GPU Memory Usage Profiling on Plain AlexNet with The Optimal Set of Checkpoints, {3, 4, 5, 9, 11, 12}.

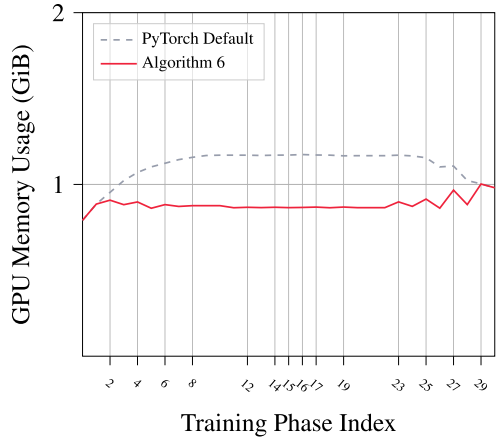


Fig. 9. GPU Memory Usage Profiling on Modified AlexNet with The Optimal Set of Checkpoints, {2, 4, 6, 8, 12, 14, 15}.

5.6.2. AlexNet: standalone max-pooling layers

In this trial, we use the extended model specs of AlexNet, i.e. it has 5 convolution layers and 3 standalone max-pooling layers, and 3 fully-connected layers. From the result in Fig. 9, we can see that the checkpoint subset {2, 4, 6, 8, 12, 14, 15} is chosen by our algorithm. We should note that since we have changed the granularity of the specs, the numbers (which are indexes) now have a different meaning. It means that the three standalone max-pooling layers (i.e. indices 2, 4, and 8) are chosen as the checkpoints.

In the first convolution layer (Conv2d/ReLU + MaxPool2d) of AlexNet, the output tensor sizes of Conv2d/ReLU and MaxPool2d are 103 MiB and 26 MiB for a batch size of 128, respectively. Without separating the max-pooling, only the tensor of size 26 MiB can be selected

Table 3
The Training Time of One Batch Using Different Methods.

Exec. time per batch (second)	Algorithm 6	Algorithm 3	ACG Solver	$O(\sqrt{n})$	PyTorch
VGG-19, b = 128	0.779 {2,4,6,9,11,14,16,19,21,23,24}	0.779 {3,6,24}	0.779 {3,6,24}	0.780 {5,10,15,20,24}	0.585
VGG-19, b = 256	1.541	1.540	1.540	1.541	1.158
VGG-19, b = 320	1.921	1.920	1.920	1.922	OOM
VGG-19, b = 384	2.292	2.289	2.289	OOM	OOM
VGG-19, b = 400	2.849	2.335	2.335	OOM	OOM
AlexNet, b = 128	0.046 {2,4,6,8,12,14,15}	0.046 {2,4,12,15}	0.046 {2,4,12,15}	0.046 {4,8,12,15}	0.038
AlexNet, b = 4096	1.206	1.206	1.206	1.216	1.058

as the checkpoint. By moving max-pooling out of the layer, the tensor of size 103 MiB becomes a selectable checkpoint. Consequently, our algorithm identifies a better solution by selecting it as a checkpoint, reducing the peak memory usage by 100 MiB compared to the plain AlexNet model. The result aligns with our expectations. We can have a lower peak memory usage if there are more candidates in the model specs as if checkpointing at the middle of an abstract, high-level layer.

This also shows the advantage of our algorithm. Because the time complexity of our algorithm is linear, the increase in model length caused by the increase in granularity will not affect the speed of our algorithm a lot. (in our cases, there will be at most 7 max-pooling layers given the image size is 224 by 224)

5.7. Comparison: the training time on different experiment settings

In this experiment, we compare the training time of VGG-19 and AlexNet with different algorithms. We train the model for one epoch on the ImageNet dataset and measure the average training time of a single batch. The results are shown in Table 3.

For VGG-19, the training time is close with all checkpoint selection methods, irrespective of the number of segments. The only exception is Algorithm 6 with a batch size of 400, which exhibits 22% longer time compared to other checkpoint selection methods. Since this paper focuses on finding the optimal checkpoints with minimal peak memory usage, we will investigate its reason in the future. Compared with PyTorch, the checkpoint selection methods incur 33% additional training time with batch sizes of 128 and 256.

As for AlexNet, since AlexNet is a relatively small model, data augmentation becomes a bottleneck and the GPU often becomes idle when waiting for the input data from the CPU. Therefore, we disable data augmentation and reuse the same input batch data when measuring the training time of AlexNet. This way avoids extra overhead and ensures full GPU utilization. The result in Table 3 indicates that the checkpointing methods require 14% more training time compared to PyTorch with a batch size of 4096.

We implemented our checkpoint selection algorithms using Python. Algorithm 3 and Algorithm 6 respectively take 20 milliseconds and 1.1 milliseconds to find the checkpoint subset for VGG-19, executed on the Intel Xeon Gold 6226R CPU.

6. Conclusion and future work

In this paper, we identify the memory pressure problem commonly seen in modern Deep Neural Network training among state-of-the-art platforms. Then we review the well-known insight proposed by Chen et al., i.e. *trading computation for memory*, and build a dynamic programming algorithm accordingly. From there, we refine the objective function for a more precise estimation of GPU memory cost to align our result with the prominent state-of-the-art deep-learning platform, PyTorch. With extensive experiments, we show that a more precise model specification can decrease peak GPU memory usage even more. While that means an increase in the input size, it is not a problem for our algorithm because it runs in linear time.

In the future, we would like to investigate the mechanism of GPU memory allocation in modern deep-learning platforms to reduce those *preserved memory* reported by Nvidia hardware. On the other hand, since we can divide an arbitrary computation graph into many linear submodels, it will be interesting to generalize our algorithm for a general DNN model, including those with thousands of layers, which is apparently not analyzable by a naive $O(n^3)$ algorithm.

CRediT authorship contribution statement

Ding-Yong Hong: Validation, Software. **Tzu-Hsien Tsai:** Methodology, Formal analysis. **Ning Wang:** Validation, Software, Methodology. **Pangfeng Liu:** Writing – original draft, Validation, Methodology, Formal analysis. **Jan-Jan Wu:** Methodology, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] O. Beaumont, L. Eyraud-Dubois, A. Shilova, Efficient combination of rematerialization and offloading for training dnns, in: Neural Information Processing Systems, 2021.
[2] T. Chen, B. Xu, C. Zhang, C. Guestrin, Training deep nets with sublinear memory cost, arXiv:1604.06174, 2016.
[3] J. Feng, D. Huang, Optimal gradient checkpoint search for arbitrary computation graphs, arXiv:1808.00079, 2021.
[4] A.N. Gomez, M. Ren, R. Urtasun, R.B. Grosse, The reversible residual network: backpropagation without storing activations, in: NIPS, 2017.
[5] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016, http://www.deeplearningbook.org.
[6] A. Gruslys, R. Munos, I. Danihelka, M. Lanctot, A. Graves, Memory-efficient backpropagation through time, in: NIPS, 2016.
[7] S. Gupta, A. Agrawal, K. Gopalakrishnan, P. Narayanan, Deep learning with limited numerical precision, in: International Conference on Machine Learning, 2015.
[8] S. Han, H. Mao, W.J. Dally, Deep compression: compressing deep neural network with pruning, trained quantization and Huffman coding, in: Computer Vision and Pattern Recognition, 2015.
[9] S. Han, J. Pool, J. Tran, W.J. Dally, Learning both weights and connections for efficient neural network, in: NIPS, 2015.
[10] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 770–778.
[11] J. Herrmann, O. Beaumont, L. Eyraud-Dubois, J. Hermann, A. Joly, A. Shilova, Optimal checkpointing for heterogeneous chains: how to train deep neural networks with limited memory, arXiv:1911.13214, 2019.
[12] J. Hu, L. Shen, S. Albanie, G. Sun, E. Wu, Squeeze-and-excitation networks, IEEE Trans. Pattern Anal. Mach. Intell. 42 (2017) 2011–2023.

[13] C. chin Huang, G. Jin, J. Li, Swapadvisor: pushing deep learning beyond the gpu memory limit via smart swapping, in: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, 2020.

[14] P. Judd, J. Albericio, T. Hetherington, T.M. Aamodt, N.E. Jerger, A. Moshovos, Proteus: exploiting numerical precision variability in deep neural networks, in: Proceedings of the 2016 International Conference on Supercomputing, Association for Computing Machinery, New York, NY, USA, 2016, https://doi.org/10.1145/2925426.2926294.

[15] M. Kirisame, S. Lyubomirsky, A. Haan, J. Brennan, M. He, J. Roesch, T. Chen, Z. Tatlock, Dynamic tensor rematerialization, in: International Conference on Learning Representations, 2021, https://openreview.net/forum?id=Vfs_2RnOD0H.

[16] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Commun. ACM 60 (2017) 84–90.

[17] T.D. Le, H. Imai, Y. Negishi, K. Kawachiya, Tflms: large model support in tensorflow by graph rewriting, arXiv:1807.02037 [abs], 2018.

[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, Pytorch: an imperative style, high-performance deep learning library, arXiv:1912.01703, 2019.

[19] B. Pudipeddi, M. Mesmakhosroshahi, J. Xi, S. Bharadwaj, Training large neural networks with constant memory using a new execution algorithm, arXiv:2002.05645 [abs], 2020.

[20] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, S.W. Keckler, vdnn: virtualized deep neural networks for scalable, memory-efficient neural network design, in: 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2016, pp. 1–13.

[21] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al., Imagenet large scale visual recognition challenge, Int. J. Comput. Vis. 115 (2015) 211–252.

[22] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, CoRR, arXiv:1409.1556, 2014.

[23] N.S. Sohoni, C.R. Aberger, M. Leszczynski, J. Zhang, C. Ré, Low-memory neural network training: a technical report, arXiv:1904.10631, 2022.

[24] C. Szegedy, S. Ioffe, V. Vanhoucke, A.A. Alemi, Inception-v4, inception-resnet and the impact of residual connections on learning, in: Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, AAAI Press, 2017, pp. 4278–4284.

[25] M. Tan, Q.V. Le, Efficientnet: rethinking model scaling for convolutional neural networks, arXiv:1905.11946 [abs], 2019.

[26] Y. Wu, M. Schuster, Z. Chen, Q.V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, L. Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J.R. Smith, J. Riesa, A. Rudnick, O. Vinyals, G.S. Corrado, M. Hughes, J. Dean, Google's neural machine translation system: bridging the gap between human and machine translation, arXiv:1609.08144 [abs], 2016.

[27] Z. Wu, C. Shen, A. van den Hengel, High-performance semantic segmentation using very deep fully convolutional networks, arXiv:1604.04339 [abs], 2016.

[28] J. Yu, Z. Wang, V. Vasudevan, L. Yeung, M. Seyedhosseini, Y. Wu, Coca: contrastive captioners are image-text foundation models, Trans. Mach. Learn. Res. 2022 (2022).

[29] S. Zagoruyko, N. Komodakis, Wide residual networks, arXiv e-prints, arXiv:1605.07146, 2016.

[30] J.Y. Zhu, T. Park, P. Isola, A.A. Efros, Unpaired image-to-image translation using cycle-consistent adversarial networks, in: 2017 IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2242–2251.

[31] B. Zoph, V. Vasudevan, J. Shlens, Q.V. Le, Learning transferable architectures for scalable image recognition, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2017, pp. 8697–8710.